Практические задания по дисциплине «Системное программное обеспечение 2»

Общие требования к программной реализации работ

Для реализации необходимо использовать язык программирования, поддерживаемый комплексом программ, разработанным в первом семестре изучения дисциплины Системное программное обеспечение, если в описании задания не сказано иное.

Исходный код лабораторных работ размещать в репозитории Gitlab факультета ПИиКТ.

Также необходимо следовать данным пунктам:

1. **«Нет» статичности.** Все структуры данных должны допускать создание множества их экземпляров.
2. **«Нет» «магическим» константам.** Все значения должны либо вычисляться из обрабатываемых программой данных, либо задаваться с помощью аргументов командной строки или конфигурационных файлов.
3. **«Нет» бесконечным циклам.** Все циклы должны иметь понятные условия выхода: не допускается использовать, например, while (true), for (; ;) и т.д.
4. **«Нет» утечке ресурсов.** Все ресурсы, которые были использованы в программе и требуют освобождения (закрытия), должны корректно освобождаться (закрываться) независимо от возникновения ошибочных ситуаций или исключений. Например, открытый файл должен быть закрыт после того, как он перестал использоваться в программе; аллоцированная вручную память обязательно должна освобождаться.
5. **«Нет» неожиданным завершениям программы.** Все процессы, нити (threads) должны корректно завершаться в результате выполнения работы, а не прерываться функциями вида Abort/Exit.
6. **«Нет» побайтовому вводу-выводу.** Все данные должны обрабатываться частями (блоками) известного размера, с учетом целесообразного размера буфера.

Настоятельно рекомендуется:

- Использовать окружение разработчика, сформированное в первом семестре изучения дисциплины Системное программное обеспечение.
- Сборку проекта осуществлять с помощью кроссплатформенных средств автоматизации, таких как мэйкфайлы.
- Следовать общим принципам грамотной разработки ПО, таким как SOLID, DRY, и др., грамотно использовать непрозрачные типы данных (opaque data types), разделять публичную и приватную функциональность модулей.

Оформление отчетов

По каждому из заданий должен быть представлен отчет, содержащий следующие части:

1. Титульный лист установленной формы, включающей следующие сведения: отчёт к практическому заданию №X по дисциплине У студента Z, группа, наименование факультета и вуза, год обучения.
2. Цели – описание цели задания (см. текст задания)
3. Задачи – путь достижения цели, что именно нужно было сделать для выполнения задания (план хода вашей работы)
4. Описание работы – внешнее описание созданной программы, состава модулей, способов её использования, примеры входной и выходной информации (модули, интерфейсы, тесты)
5. Аспекты реализации – внутреннее описание созданной программы, особенности алгоритмов, примеры кода
6. Результаты – что было сделано для выполнения задач кратко по пунктам (созданные артефакты, результаты тестов, количественные оценки)
7. Выводы – что было достигнуто в отношении цели задания. (что показали тесты, почему, как это было достигнуто, чему научились, качественные оценки)

Описание заданий

Задание 1

Используя комплекс программ, разработанный в первом семестре изучения дисциплины Системное программное обеспечение, реализовать алгоритмы планирования задач в многопоточной среде по варианту. Реализовать тестовую программу и показать различия в ходе её выполнения при запуске под управлением различных алгоритмов планирования.

Порядок выполнения:

1. Изучить порядок работы с таймером в среде выполнения программы (учебной или реальной ВМ).

2. Реализовать процедуры, необходимые для переключения контекста CPU между различными логическими потоками управления.

3. Изучить и реализовать заданные алгоритмы планирования задач, используя процедуры из п.2 и таймер в качестве триггера для вызова планировщика.

4. Реализовать тестовую программу, эмулирующую вычислительную нагрузку с помощью заданного алгоритма.

5. Запустить тестовую программу под управлением различных алгоритмов планирования задач и собрать информацию о ходе её работы.

6. Подготовить отчёт, содержащий результаты выполнения пунктов по порядку.

7. Согласовать содержание отчёта с преподавателем, при необходимости внести правки.

Задание 2

Изучить механизмы синхронизации и разработать реализацию объектов синхронизации, необходимых для упорядочивания доступа к структурам данных. Реализовать объект последовательного потока данных и показать корректность его функционирования с помощью тестовой программы по варианту.

1. Изучить основные объекты синхронизации и ассоциированную с ними внутреннюю логику.

2. Модифицировать алгоритм планирования задач в многопоточной среде, реализованный в задании 1, для поддержки объектов синхронизации и возможности пассивного ожидания по запросу.

3. Реализовать объект последовательного потока данных.

4. Реализовать тестовую программу для алгоритмов по варианту, используя объекты последовательного потока данных.

5. Запустить тестовую программу под управлением различных алгоритмов планирования задач и собрать информацию о ходе её работы.

6. Подготовить отчёт, содержащий результаты выполнения пунктов по порядку.

7. Согласовать содержание отчёта с преподавателем, при необходимости внести правки.

Задание 3

Используя комплекс программ, разработанный в первом семестре изучения дисциплины Системное программное обеспечение, реализовать поддержку набора операций извлечения данных для структуры данных по варианту.

1. Изучить внутреннюю организацию структуры, заданной данных и существующие инструменты для инспектирования и формирования её состояния.

2. Разработать набор функций, осуществляющих инспектирование и извлечение данных из образа заданной структуры данных в памяти с использованием объектов последовательного потока данных, реализованного в задании 2.

4. Реализовать тестовую программу, использующую набор функций из п.2 для предоставления пользователю интерфейса к структуре данных, содержащейся в заданном файле образа.

3. Подготовить тестовый образ структуры данных по согласованию с преподавателем.

4. Показать корректность реализованного в п.2 набора функций за счёт демонстрации работы тестовой программы.

5. Подготовить отчёт, содержащий результаты выполнения пунктов по порядку.

6. Согласовать содержание отчёта с преподавателем, при необходимости внести правки.

Варианты к заданию 1

Существует достаточно большой набор разнообразных алгоритмов планирования, которые предназначены для достижения различных целей и эффективны для разных классов задач.

Рассмотрим следующие.

First-Come, First-Served (FCFS)

Простейшим алгоритмом планирования является алгоритм, который принято обозначать аббревиатурой FCFS по первым буквам его английского названия – First-Come, First-Served (первым пришел, первым обслужен). Представим себе, что процессы, находящиеся в состоянии готовность, выстроены в очередь. Когда процесс переходит в состояние готовность, он, а точнее, ссылка на его PCB помещается в конец этой очереди. Выбор нового процесса для исполнения осуществляется из начала очереди с удалением оттуда ссылки на его PCB. Очередь подобного типа имеет в программировании специальное наименование – FIFO, сокращение от First In, First Out (первым вошел, первым вышел).

Такой алгоритм выбора процесса осуществляет невытесняющее планирование. Процесс, получивший в свое распоряжение процессор, занимает его до истечения. После этого для выполнения выбирается новый процесс из начала очереди.

Круговое, карусельное, Round Robin (RR)

Модификацией алгоритма FCFS является алгоритм, получивший название Round Robin (Round Robin – это вид детской карусели в США) или сокращенно RR. По сути дела, это тот же самый алгоритм, только реализованный в режиме вытесняющего планирования. Можно представить себе все множество готовых процессов организованным циклически – процессы сидят на карусели. Карусель вращается так, что каждый процесс находится около процессора небольшой фиксированный квант времени, обычно 10 – 100 миллисекунд. Пока процесс находится рядом с процессором, он получает процессор в свое распоряжение и может исполняться. Реализуется такой алгоритм так же, как и предыдущий, с помощью организации процессов, находящихся в состоянии готовность, в очередь FIFO. Планировщик выбирает для очередного исполнения процесс, расположенный в начале очереди, и устанавливает таймер для генерации прерывания по истечении определенного кванта времени. При выполнении процесса возможны два варианта.

- Время непрерывного использования процессора, необходимое процессу, меньше или равно продолжительности кванта времени. Тогда процесс по своей воле освобождает процессор до истечения кванта времени, на исполнение поступает новый процесс из начала очереди, и таймер начинает отсчет кванта заново.
- Продолжительность остатка текущего CPU процесса больше, чем квант времени. Тогда по истечении этого кванта процесс прерывается таймером и помещается в конец очереди процессов, готовых к исполнению, а процессор выделяется для использования процессу, находящемуся в ее начале.

Shortest process next (SPN)

При рассмотрении алгоритмов FCFS и RR видно, насколько существенным для них является порядок расположения процессов в очереди процессов, готовых к исполнению. Если короткие задачи расположены в очереди ближе к ее началу, то общая производительность этих алгоритмов значительно возрастает. Если бы мы знали время исполнения следующих процессов, находящихся в состоянии готовность, то могли бы выбрать для исполнения не процесс из начала очереди, а процесс с минимальной длительностью. Если же таких процессов два или больше, то для выбора одного из них можно использовать уже известный нам алгоритм FCFS. Квантование времени при этом не применяется. Описанный алгоритм получил название "кратчайшая работа первой

SPN -алгоритм невытесняющий. При невытесняющем SPN – планировании процессор предоставляется избранному процессу на все необходимое ему время, независимо от событий, происходящих в вычислительной системе.

Shortest remaining time (SRT)

SRT – вытесняющий вариант планирования. В этом варианте планирования планировщик выбирает процесс с наименьшим остающимся временем. Учитывается появление новых процессов в очереди готовых к исполнению (из числа вновь родившихся или разблокированных) во время работы выбранного процесса. Если время работы вновь появившегося процесса меньше, чем остаток времени выполнения исполняющегося, то исполняющийся процесс вытесняется новым.

См. Столлингс В. Операционные системы. Внутреннее устройство и реализация. Часть 4.

В соответствии с вариантом задания реализовать алгоритм планирования (2 шт.).

Программно подготовить исходные данные, представляющие собой пары чисел, где первое число – время поступления процесса, а второе – время его выполнения. Очевидно, что последовательность первых значений – возрастающая (временные парадоксы запрещены).

Например:

время появления:	0	4	6	7	11	13	19	24
длительность выполнения	4	7	8	4	5	8	2	5

Диапазон времен выполнения, и среднее время между поступающими процессами взять из варианта задания.

Число пар <время поступления> – <длительность выполнения > не менее двадцати.

Для полученных данных определить среднее время оборота процессов и среднее время ожидания процессов, в соответствии с реализованными алгоритмами планирования.

Сделать вывод о предпочтительном алгоритме планирования, считая критерием эффективности минимальное среднее время оборота процессов для нечетных вариантов, и минимальное среднее время ожидания для четных вариантов.

Вариант	Алгоритм 1	Алгоритм 2	Диапазон длительностей выполнения	Среднее время между поступающими процессами.
1	FCFS	RR(2)	4-8	6, 3
2	SPN	RR(4)	5-12	5, 3
3	SRT	SPN	3-9	4, 3
4	FCFS	SPN	3-7	3, 3
5	FCFS	SRT	2-10	5, 4
6	RR(4)	SPN	3-12	6, 3
7	SRT	RR(4)	8-15	7, 3
8	FCFS	SRT	2-12	8, 3
9	RR(2)	SRT	4-8	6, 4
10	FCFS	SPN	3-10	5, 3
11	SPN	RR(1)	3-6	4, 3
12	SRT	RR(3)	4-8	5, 3
13	FCFS	RR(5)	4-11	2, 7
14	RR(2)	SRT	3-11	7, 3
15	SRT	RR(2)	4-8	6, 4
16	FCFS	RR(4)	6-12	7, 3
17	FCFS	RR(1)	3-12	10, 3
18	SRT	RR(3)	4-10	10, 5
19	RR(2)	SRT	4-8	6, 3
20	FCFS	SPN	4-8	6, 3

Варианты к заданию 2

Пройти по ссылке https://se.ifmo.ru/courses/dbis и использовать номер варианта в разделе «Лабораторная работа #3» для получения описаний выборок. Каждую выборку декомпозировать в терминах map-reduce, представив реляционные операторы и операторы над множеством записей как самостоятельные процессы-обработчики, каждый из которых получает и порождает потоки элементов данных по мере необходимости.

Реализовать каждый этап обработки как самостоятельную процедуру, получающую входные элементы данных через последовательные байтовые потоки и передающую результирующие элементы данных через исходящие последовательные байтовые потоки. Все необходимые потоки и константные параметры для этапа обработки (значения для сравнений и т.п.) выступают в качестве параметров соответствующих процедур-обработчиков. Реализовать каждую выборку по варианту как основную процедуру программы, открывающую входящие потоки данных из выданного набора файлов данных для обработки, поток данных для вывода результата и запускающую ряд необходимых процедур-обработчиков в отдельных потоках управления, передавая им в том числе все константные параметры выборки. Предусмотреть возможность изменения количества потоков управления, инициируемых для обработки данных.

Подобрать значения константных параметров выборки для получения непустых результатов и показать их корректность посредством сравнения с результатами выполнения аналогичных SQL запросов в существующей СУБД. Использовать любой алгоритм вытесняющего планирования потоков управления с пассивным ожиданием для обеспечения конкурентного выполнения процедур обработчиков данных, показать конкурентность в решении. Для поддержки пассивного ожидания в неблокирующем режиме использовать групповое ожидание.

Вариант			Режим
1	31	61	Синхронный, асинхронный
2	32	62	Синхронный, неблокирующий
3	33	63	Синхронный, асинхронный
4	34	64	Синхронный, неблокирующий
5	35	65	Синхронный, асинхронный
6	36	66	Синхронный, неблокирующий
7	37	67	Синхронный, асинхронный
8	38	68	Синхронный, неблокирующий
9	39	69	Синхронный, асинхронный
10	40	70	Синхронный, неблокирующий
11	41	71	Синхронный, асинхронный
12	42	72	Синхронный, неблокирующий
13	43	73	Синхронный, асинхронный
14	44	74	Синхронный, неблокирующий
15	45	75	Синхронный, асинхронный
16	46	76	Синхронный, неблокирующий
17	47	77	Синхронный, асинхронный
18	48	78	Синхронный, неблокирующий
19	49	79	Синхронный, асинхронный
20	50	80	Синхронный, неблокирующий
21	51	81	Синхронный, асинхронный
22	52	82	Синхронный, неблокирующий
23	53	83	Синхронный, асинхронный
24	54	84	Синхронный, неблокирующий
25	55	85	Синхронный, асинхронный
26	56	86	Синхронный, неблокирующий
27	57	87	Синхронный, асинхронный
28	58	88	Синхронный, неблокирующий
29	59	89	Синхронный, асинхронный
30	60	90	Синхронный, неблокирующий

Пример основной процедуры

```
void main() {

  Pipe rawXxxPipe = openPipeDevice(0x010000);
  Pipe rawYyyPipe = openPipeDevice(0x020000);

  Pipe parsedXxxPipe = createPipe();
  Pipe parsedYyyPipe = createPipe();
  Pipe filteredXxx1 = createPipe();
  Pipe filteredXxx2 = createPipe();
  Pipe filteredYyyPipe = createPipe();
  Pipe joinedXxxYyy = createPipe();

  Pipe resultsPipe = openPipeDevice(0x030000);

  createThread(SheetsParserProc, new InAndOutStreamsThreadParams(
    rawXxx.getInputStream(), parsedXxx.getOutputStream()
  ));

  createThread(SheetsTypesParserProc, new InAndOutStreamsThreadParams(
    rawYyyPipe.getInputStream(), parsedYyyPipe.getOutputStream()
  ));

  craeteThread(SheetsByPersonIdFilterProc, new SheetsByPersonIdFilterThreadParams(
    parsedXxx.getInputStream(), filteredXxx1.getOutputStream(), '<', 153285)
  ));
  craeteThread(SheetsByPersonIdFilterProc, new SheetsByPersonIdFilterThreadParams(
    filteredXxx1.getInputStream(), filteredXxx2.getOutputStream(), '>', 142390)
  ));

  craeteThread(YyyByNameFilterProc, new SheetTypeByNameFilterThreadParams(
    parsedYyyPipe.getInputStream(), filteredYyyPipe.getOutputStream(), '<', "Перезачет")
  ));

  createThread(JoinSheetsToYyyProc, new JoinSheetsToYyyThreadParams(
    filteredYyyPipe.getInputStream(), filteredXxx2.getInputStream(),
    joinedXxxYyy.getOutputStream()
  ));

  createThread(ResultsFormatterProc, new ResultsFormatterThreadParams(
    joinedXxxYyy.getInputStream(), resultsPipe.getOutputStream()
  ));

  // wait for all the work to be done and then exit from main()
  // to the entry point containing HLT instruction terminating the machine
}
```

Варианты к заданию 3

Общий алгоритм работы программы:

1. Проверить, поддерживается ли файловая система на заданном разделе или диске.
2. В случае, если файловая система поддерживается, программа переход в диалоговый режим с ожиданием ввода команд от пользователя в соответствии с протоколом PASSIVE FTP. Подержать основные команды, необходимые для чтения информации с помощью существующих FTP-клиентов:
    a. вывод списка имен и атрибутов элементов указанной директории;
    b. копирование файлов или директорий из исследуемой (заданной по варианту) файловой системы;
    c. отображение названия «текущей» директории и переход в другую директорию.

Вариант	ФС	Источники документации
1	FAT32	https://docs.microsoft.com/en-us/previous-versions/windows/it-pro/windows-server-2003/cc776720(v=ws.10) https://en.wikipedia.org/wiki/Design_of_the_FAT_file_system
2	NTFS	https://docs.microsoft.com/en-us/previous-versions/windows/it-pro/windows-server-2003/cc758691(v=ws.10)?redirectedfrom=MSDN **Книга:** Windows NT File System Internals: A Developer's Guide Paperback – September 11, 1997
3	Ext3	https://students.mimuw.edu.pl/SO/Linux-doc/UnderstLinuxKernel2e-ch17.pdf **Книга:** Understanding the Linux Kernel Daniel P. Bovet and Marco CesatI
4	Btrfs	https://btrfs.wiki.kernel.org/index.php/Btrfs_design https://btrfs.wiki.kernel.org/index.php/On-disk_Format https://btrfs.wiki.kernel.org/index.php/Data_Structures
5	HFS Plus	https://developer.apple.com/library/archive/technotes/tn/tn1150.html https://github.com/libyal/libfshfs/blob/main/documentation/Hierarchical%20File%20System%20(HFS).asciidoc
6	Reiser	http://jcoppens.com/univ/data/pdf/fs/reiserfs.pdf https://reiser4.wiki.kernel.org/index.php/Main_Page
7	Xfs	http://ftp.ntu.edu.tw/linux/utils/fs/xfs/docs/xfs_filesystem_structure.pdf https://xfs.org/docs/xfsdocs-xml-dev/XFS_Filesystem_Structure/tmp/en-US/html/index.html
8	UFS/FFS	https://flylib.com/books/en/2.849.1.110/1/ http://callibra.com.br/Violino/Carrier,%20Brian%20-%20File%20System%20Analysis.pdf